

# **INFORME DE INGENIERÍA DE SOFTWARE: SISTEMA DE PLANIFICACIÓN Y OPTIMIZACIÓN DE RUTINAS DEPORTIVAS**

**Universidad:** Universidad Andrés Bello.

**Facultad:** Facultad de Ingeniería.

**Carrera:** Ingeniería en Computación e Informática.

**Asignatura:** Paradigmas de Programación.

**Fecha:** 28 de mayo 2026.

**NRC:** 8025.

**Ciudad:** Viña del Mar, Chile.

**Integrantes:**

- Jordán Acuña, 21.682.867-4
- Kevin Soto, 21.092.483-3

## **1. INTRODUCCIÓN.**

En el ámbito del entrenamiento de alto rendimiento y la preparación física general, la dosificación de la carga de trabajo y la variabilidad de los estímulos son pilares fundamentales para evitar el estancamiento motor y la fatiga crónica. La planificación manual de rutinas presenta una alta propensión a errores humanos, tales como la duplicación de ejercicios demandantes de forma consecutiva o la asignación incorrecta de niveles de intensidad.

El presente proyecto aborda esta problemática mediante el desarrollo de una solución de software de escritorio robusta en el lenguaje de programación Java. El sistema automatiza el procesamiento de catálogos deportivos extensos almacenados en archivos planos, aplicando restricciones algorítmicas temporales y de categorización para generar planes de entrenamiento optimizados y seguros para el usuario final.

## **2. OBJETIVOS DEL PROYECTO.**

### **2.1. Objetivo General**

- Diseñar e implementar un sistema de software modularizado y concurrente que automatice la lectura, validación y filtrado de un catálogo de ejercicios deportivos para la generación de rutinas personalizadas bajo restricciones físicas y temporales estrictas.

## 2.2. Objetivos Específicos

- Garantizar la separación de responsabilidades mediante una arquitectura orientada a paquetes (*model, service, exception, views*).
- Implementar un motor de lectura de archivos planos capaz de *parsear* de forma segura datos estructurados e ignorar metadatos o comentarios.
- Desarrollar un algoritmo de filtrado *multi-condicional* que valide la intensidad, respete los cupos por tipo de ejercicio y aplique la restricción de descarte por semanas consecutivas.
- Diseñar una interfaz gráfica de usuario (GUI) fluida y *asincrónica* que mitigue el bloqueo del hilo principal de ejecución utilizando el patrón de diseño *Observer*.

## 3. REQUERIMIENTOS DEL SISTEMA

A continuación, se dejan por escrito las especificaciones técnicas evaluadas y cubiertas por la aplicación:

- **Requerimiento Funcional 01:** El sistema debe leer y parsear la información del archivo plano ejercicios.txt.
- **Requerimiento Funcional 02:** Validar el formato interno del archivo, rechazando registros incompletos o corruptos.
- **Requerimiento Funcional 03:** Filtrar los ejercicios de acuerdo con la intensidad exacta provista por el usuario.
- **Requerimiento Funcional 04:** Aplicar la restricción de exclusión si el ejercicio fue utilizado en la semana actual o anterior.
- **Requerimiento Funcional 05:** Calcular resúmenes estadísticos automáticos (tiempo total y conteo por tipo).
- **Requerimiento No Funcional 01:** Arquitectura modularizada con aislamiento de lógica de negocio y vistas.
- **Requerimiento No Funcional 02:** Persistencia ligera basada exclusivamente en archivos de texto delimitados por punto y coma.
- **Requerimiento No Funcional:** Procesamiento asincrónico multitarea para la manipulación de archivos y cálculo algorítmico.

## 4. DISEÑO ARQUITECTÓNICO Y MODULARIZACIÓN

Para cumplir con las exigencias de mantenibilidad y escalabilidad propias de la ingeniería de software, el código fuente se estructuró de forma jerárquica en los siguientes paquetes bajo la raíz `backend` y `frontend`:

- *backend.exception*: Define excepciones personalizadas (*GimnasioException*) para controlar flujos inesperados, como la ausencia del archivo de texto o formatos de datos corruptos.
- *backend.model*: Contiene las entidades principales del dominio del problema (Ejercicio, Rutina) y las enumeraciones que restringen sus atributos (*TipoEjercicio*, *NivelIntensidad*).
- *backend.service*: Aloja el motor lógico del sistema (*GimnasioBackend*). Se encarga de procesar el archivo plano, aplicar los filtros lógicos matemáticos y construir las rutinas en memoria. También expone la interfaz *GimnasioListener* para el manejo de eventos.
- *frontend.views*: Contiene la interfaz gráfica de usuario (*MainFrame*) construida con la librería estándar *javax.swing*, separando completamente la vista de la lógica de negocio.

## 5. LÓGICA ALGORÍTMICA Y MATEMÁTICA

### 5.1. Mecanismo de Parseo de Archivos Planos

La lectura de datos se realiza a través de un flujo estructurado con *BufferedReader* y *FileReader*. El algoritmo procesa el archivo línea por línea ejecutando las siguientes acciones de filtrado de texto:

1. **Limpieza de Espacios:** Se aplica *.trim()* para eliminar caracteres invisibles de control.
2. **Ignorado de Comentarios:** Si la línea inicia con el token `#` o se encuentra vacía, el sistema ejecuta una instrucción `continue` para saltar directamente al siguiente registro.
3. **Tokenización:** Se divide la cadena usando el delimitador punto y coma (`;`) mediante *.split(";")*.
4. **Validación de Robustez:** Si el arreglo resultante posee una longitud menor a 7 columnas, el sistema aborta la operación lanzando una excepción controlada para evitar un desborde de índice (*ArrayIndexOutOfBoundsException*).

## 5.2. Algoritmo de Selección Deportiva y Restricción Temporal

La generación de la rutina óptima sigue un criterio restrictivo estricto. Un ejercicio es considerado **elegible** si y solo si cumple de forma simultánea con el siguiente conjunto de condiciones lógicas:

1. **Filtro de Intensidad:** El ejercicio evaluado debe coincidir exactamente con la intensidad solicitada por el usuario (ej. INTERMEDIO, AVANZADO).
2. **Restricción Temporal:** Se evalúa la semana del último uso del ejercicio. Si se utilizó en la semana actual (\$W\$) o en la semana inmediatamente anterior (\$W - 1\$), el ejercicio es descartado automáticamente para prevenir la fatiga muscular del usuario.
3. **Cupos Disponibles:** Se clasifica y añade el ejercicio a la rutina final únicamente si aún existen cupos vacantes para su categoría específica (Fuerza o Cardiovascular).

## 6. CONCURRENCIA Y CONTROL DE EXCEPCIONES

### 6.1. Procesamiento Multihilo de la Interfaz (Asincronía)

Las operaciones que involucran acceso a disco duro o iteraciones sobre grandes volúmenes de datos suelen ralentizar el hilo principal de la interfaz gráfica (conocido en Java como el *Event Dispatch Thread* o EDT), provocando el congelamiento visual de las ventanas.

Para resolver este problema, el sistema delega las tareas pesadas a subprocesos independientes instanciando objetos de tipo Thread. Una vez procesados los datos en el hilo secundario, el backend invoca los métodos de la interfaz GimnasioListener. La vista captura este evento y utiliza la instrucción `SwingUtilities.invokeLater()` para retornar el control de forma segura al EDT, actualizando los elementos gráficos sin generar colisiones de memoria o inconsistencias visuales.

### 6.2. Arquitectura de Excepciones Personalizada

El sistema no permite que los errores nativos de Java (como *FileNotFoundException* o *NumberFormatException*) se muestren directamente al usuario de forma cruda. En su lugar, todas las fallas previsibles se capturan dentro de bloques try-catch y se unifican bajo la clase especializada *GimnasioException*. Esto permite centralizar los mensajes de error en cuadros de diálogo claros y profesionales controlados por el método `onErrorOcurrido(String mensaje)`.

## 7. CONCLUSIÓN

La arquitectura implementada en el proyecto demuestra la viabilidad de construir sistemas de escritorio robustos, mantenibles y altamente eficientes utilizando herramientas estándar de Java. La correcta separación en capas asegura que modificaciones futuras en las reglas del gimnasio o cambios en el formato visual no alteren el core algorítmico del backend. El cumplimiento de las restricciones temporales automatizadas dota a la aplicación de un valor agregado real, garantizando la integridad física de los usuarios mediante el uso inteligente de la lógica de programación.